Core Java (OOPS)

1. Abstract Class and Interface?
2. class and interfaces in java?
3. String & string buffer, String Builder
4. Immutable class and how to achieve it and types of Inheritance?
5. Immutable class implementation?
6. All Oops concepts?
7. Shallow cloning and deep cloning?
8. Which is better for concat strings : string or string buffer or string builder?
9. How to handle infinity in java?
10. How to filter Strings Object from Arraylist is multiple type object are there.(IBM)?
11. Lazy Loading in stream
12. Sort Circuit in stream.
13. How to JVM Work?
14. How does the JVM identify objects for Garbage Collection?

    Ans. The JVM uses a process called **Reachability Analysis**.

    The Garbage Collector starts from special root references called **GC Roots** and checks whether an object can still be reached.

    GC Roots include:

    - Local variables in active stack frames
    - Static variables
    - Active threads
    - JNI (Native) references

15. Can we extends String class?

    Ans. No because it is final

16. Can we change the override method scope or <u>Changing visibility of method in inherited class</u>?

Ans: It is not possible to reduce the visibility of a method in subclass. The subclass need to be a valid instance of the base class.

Parent Class ⇓ Scope	public	protected	default	Private
Public	Y	N	N	N
Protected	Y	Y	N	N
Default	Y	Y	Y	N
Private (Not access)				

17. Can we extends Static class?

Ans. extending static classes is allowed, since its members are not necessarily static. the static modifier can only be used on nested classes because it can only be used on class members (and only nested classes can be class members).

18. what is mean of native library or native code?

Ans. A native library is a library that contains "native" code. That is, code that has been compiled for a specific hardware architecture or operating system such as x86 or windows. Including such native library in your project may break the platform-independence of you application.

"Native Library" generally means a non-Java library that's used by the system (so C/C++, etc). Think normal DLLs or libs.

Java can load these native libraries through JNI.

19. vertical scaling vs horizontal scaling?

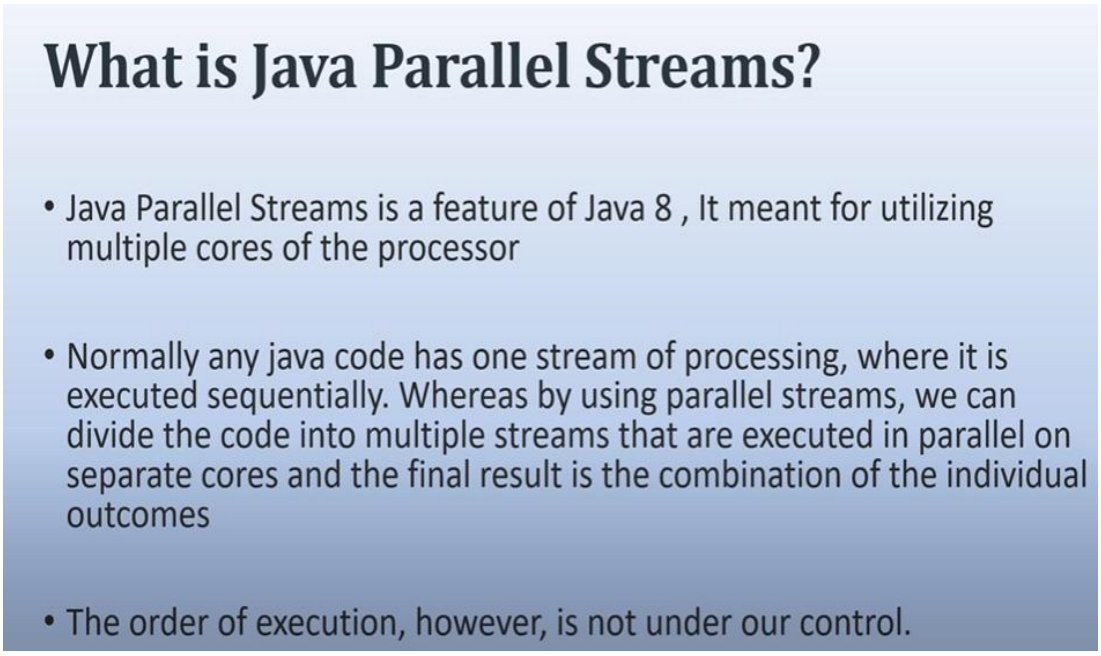
The main difference between scaling up and scaling out is that horizontal scaling simply adds more machine resources to your existing machine infrastructure. Vertical scaling adds power to your existing machine infrastructure by increasing power from CPU or RAM to existing machines.

20. Difference between String and StringBuffer and String Builder?

No.	String	StringBuffer
1)	The String class is immutable.	The StringBuffer class is mutable.
2)	String is slow and consumes more memory when we concatenate too many strings because every time it creates new instance.	StringBuffer is fast and consumes less memory when we concatenate t strings.
3)	String class overrides the equals() method of Object class. So you can compare the contents of two strings by equals() method.	StringBuffer class doesn't override the equals() method of Object class.
4)	String class is slower while performing concatenation operation.	StringBuffer class is faster while performing concatenation operation.
5)	String class uses String constant pool.	StringBuffer uses Heap memory

No.	StringBuffer	StringBuilder
1)	StringBuffer is *synchronized* i.e. thread safe. It means two threads can't call the methods of StringBuffer simultaneously.	StringBuilder is *non-synchronized* i.e. not thread safe. It means two threads can call the methods of StringBuilder simultaneously.
2)	StringBuffer is *less efficient* than StringBuilder.	StringBuilder is *more efficient* than StringBuffer.
3)	StringBuffer was introduced in Java 1.0	StringBuilder was introduced in Java 1.5

21. ParallelStream:
https://www.youtube.com/watch?v=J7YqYIaev7g&ab_channel=JavaTechie



22. Difference between Function & Predicate?

Sr. No.	Key	Function	Predicate
1	Basic	It can take 2 type parameters First one represents input type argument type and second one represents return type.	It can take one type parameter which represents input type or argument type.
2	Return Type	It can return any type of value.	It can only return boolean value
3	Method	It has abstract method apply().	It has abstract method test().

Sr. No.	Key	Function	Predicate
4.	Use Case	It can be used to implement conditional checks	It can be used for the transformation and to the return values.

```
public class Main {
  public static void main(String args[]) {
    List<Integer> numList = new ArrayList<>();
    numList.add(5);
    numList.add(10);
    Predicate<Integer> pred = i -> i > 5;
    numList.stream().filter(pred).forEach(i -> System.out.println(i));
  }
}
```

```
public class Main {
  public static void main(String args[]) {
    List<Integer> numList = new ArrayList<>();
    numList.add(78);
    numList.add(10);
    Function<Integer, Integer> fun = i -> i / 2;
    numList.stream().map(fun).forEach(System.out::println);
  }
}
```

23. Stream.reduce() in Java (https://www.geeksforgeeks.org/stream-reduce-java-examples/)

The Stream.reduce() method is used to perform a reduction operation on the elements of a stream using an associative accumulation function and returns an Optional. It is commonly used to aggregate or combine elements into a single result, such as computing the maximum, minimum, sum, or product.

```
class GFG {
    public static void main(String[] args) {
        // Creating a list of Strings
        List<String> words = Arrays.asList("GFG", "Geeks", "for", "GeeksQuiz", "GeeksforGeeks");
```

```
        // Using reduce to find the longest string in the list
        Optional<String> longestString = words.stream()
            .reduce((word1, word2) -> word1.length() > word2.length() ? word1 : word2);

        // Displaying the longest String
        longestString.ifPresent(System.out::println);
    }
}
```

24. Why use SerialVersionUID in Java? And difference between our own serializable id and jvm created?

Ans. It is because SerialVersionUID is used to ensure that during deserialization the same class (that was used during serialize process) is loaded.

Ex: private static final long serialVersionUID = 1L;

Custom serialVersionUID	JVM-generated serialVersionUID
Explicitly controlled by developer	Generated automatically by JVM
Remains same until developer changes it	Changes whenever class structure changes
Provides backward compatibility	Can break compatibility unexpectedly
Recommended in production	Not recommended for long-term storage

25. Serializable example.https://www.youtube.com/watch?v=Ki4c3hmS-6I

26. What is new memory in java 8?

The following table describes the difference between metaspace and PermGen:

PermGen	MetaSpace
It is removed from java 8.	It is introduced in Java 8.
PermGen always has a fixed maximum size.	Metaspace by default auto increases its size depending on the underlying OS.

PermGen	MetaSpace
Contiguous Java Heap Memory.	Native Memory(provided by underlying OS).
Inefficient garbage collection.	Efficient garbage collection.

27. Difference between Syncronized hashmap and concurrent hashmap?

Ans:

ConcurrentHashMap	Synchronized HashMap
ConcurrentHashMap is a class that implements the ConcurrentMap and serializable interface.	We can synchronize the HashMap by using the **synchronizedMap()** method of java.util.Collections class.
It locks some portion of the map.	It locks the whole map.
ConcurrentHashMap allows performing concurrent read and write operation. Hence, performance is relatively better than the Synchronized Map.	In Synchronized HashMap, multiple threads can not access the map concurrently. Hence, the performance is relatively less than the ConcurrentHashMap.
ConcuurentHashMap doesn't allow inserting null as a key or value.	Synchronized HashMap allows inserting null as a key.
ConccurentHashMap doesn't throw ConcurrentModificationException.	Synchronized HashMap throw **ConcurrentModificationException**.

28. Memory MetaSpace in Java 8?

PermGen	MetaSpace
It is removed from java 8.	It is introduced in Java 8.
PermGen always has a fixed maximum size.	Metaspace by default auto increases its size depending on the underlying OS.

Contiguous Java Heap Memory.	Native Memory(provided by underlying OS).
Inefficient garbage collection.	Efficient garbage collection.

29. Load Factor in HashMap and What happens when Load Factor is exceeded?

(https://www.javatpoint.com/load-factor-in-hashmap)

Load Factor determines **when a HashMap should increase its capacity**

Load Factor = Number of Entries / Current Capacity
Default load factor in Java: 0.75f
Initial Capacity = 16

threshold = capacity * loadFactor
Threshold: 16 × 0.75 = 12
When size exceeds 12: resize();

When the **13th element** is inserted:

- Capacity increases from 16 → 32
- Existing entries are rehashed and redistributed

Note: New Capacity: 16 → 32 → 64 → 128 (HashMap always doubles the capacity)

Load Factor is the threshold that decides when a HashMap should resize. By default it is 0.75, meaning the HashMap resizes when 75% of its buckets are filled, balancing memory usage and lookup performance.

When the number of entries exceeds: capacity × loadFactor
HashMap performs:

1. Capacity doubled
2. New bucket array created
3. Existing entries rehashed
4. Entries moved to new buckets

This process is called **Rehashing (Resizing)**.

30. Difference callable and Runnable interface?

Ans. The main difference between the Callable and Runnable interfaces in Java is that Callable can return a value, while Runnable cannot:

- **Callable** Returns the result of executing the task to the caller. It can also throw a checked exception. Callable instances can only be executed via ExecutorService.
- **Runnable** Does not return a result and cannot throw a checked exception. Runnable instances can be run by Thread class as well as ExecutorService.

31. optional class?

32. Can we override the varags method?

Void test(a1,a2,a3,……){}
Void test(a1,a2,…){}

Ans. Yes, **varargs methods can be overloaded**, but you need to understand how Java resolves them

```
class Test {
    void test(String a1, String a2) {
        System.out.println("2 arguments");
    }
    void test(String... args) {
        System.out.println("Varargs");
    }
    public static void main(String[] args) {
        Test t = new Test();
        t.test("A", "B");        // 2 arguments
        t.test("A", "B", "C");   // Varargs
    }
}
```

33. runAsync vs supplyAsync in completableFuture?

Ans. runAsync() is used for asynchronous tasks that do not return a result (Runnable -> CompletableFuture<Void>), whereas supplyAsync() is used for asynchronous tasks that return a value (Supplier<T> -> CompletableFuture<T>).
runAsync(): Use when you want to execute a task asynchronously and **don't need any result**.
supplyAsync(): Use when you want to execute a task asynchronously and **return a value**.

Feature	**runAsync()**	**supplyAsync()**
Takes	Runnable	Supplier<T>
Returns value?	❌ No	✅ Yes
Return Type	CompletableFuture<Void>	CompletableFuture<T>
Use Case	Fire-and-forget task	Compute and return result
Functional Interface	Runnable	Supplier

34. What is CompletableFuture?

A **CompltableFuture** is used for asynchronous programming. Asynchronous programming means writing non-blocking code. It runs a task on a separate thread than the main application thread and notifies the main thread about its progress, completion or failure.

In this way, the main thread does not block or wait for the completion of the task. Other tasks execute in parallel. Parallelism improves the performance of the program.

A CompletableFuture is a class in Java. It belongs to java.util.cocurrent package. It implements CompletionStage and Future interface.

CompletionStage

- It performs an action and returns a value when another completion stage completes.
- A model for a task that may trigger other tasks.

35. Future vs. CompletableFuture?

Ans.

- A CompletableFuture is an extension to Java's Future API which was introduced in Java 8.
- A Future is used for asynchronous Programming. It provides two methods, isDone() and get(). The methods retrieve the result of the computation when it completes.
- Limitations of the Future
- A Future cannot be mutually complete.
- We cannot perform further action on a Future's result without blocking.
- Future has not any exception handling.
- We cannot combine multiple futures.
- **supplyAsync():** It complete its job asynchronously. The result of supplier is run by a task from ForkJoinPool.commonPool() as default. The supplyAsync() method returns CompletableFuture on which we can apply other methods.
- **thenApply():** The method accepts function as an arguments. It returns a new CompletableStage when this stage completes normally. The new stage use as the argument to the supplied function.
- **join():** the method returns the result value when complete. It also throws a CompletionException (unchecked exception) if completed exceptionally.

36. Can we declare the main in super and sub class both when super class extends in child?

Yes, **we can declare a main() method in both parent (super) and child (sub) classes.**
Because main() is **static**, it is **not overridden**, it is **hidden**.
So this is **method hiding**, not method overriding.

```
class Parent {
    public static void main(String[] args) {
```

```
        System.out.println("Parent Main");
    }
}
class Child extends Parent {
    public static void main(String[] args) {
        System.out.println("Child Main");
    }
}
```

37. Statefull and Stateless Operation in Java 8?

Ans: In Java 8 Streams, operations are classified as either *stateless* or *stateful*. Stateless operations process each element independently, while stateful operations require knowledge of other elements in the stream (past or future) to produce results.

Stateless Stream Operations

- **Definition:** Do not retain information about previously processed elements.
- **Characteristics:** Each element is processed independently.
- **Examples:**
  - map() → transforms each element.
  - filter() → selects elements based on a condition.
  - flatMap() → flattens nested structures.
  - peek() → performs side-effects like logging.
- **Performance:** Highly efficient and parallel-friendly because no global state is needed.

Stateful Stream Operations

- **Definition:** Require maintaining state across multiple elements to produce results.
- **Characteristics:** Depend on the collective state of the stream, not just individual elements.
- **Examples:**
  - sorted() → needs to see all elements before ordering.
  - distinct() → must track seen elements to remove duplicates.
  - limit(n) → must count elements to stop after n.
  - skip(n) → must track how many elements have been skipped.
- **Performance:** More expensive, especially in parallel streams, because they require coordination and memory to maintain state.

	Stateless Operations	**Stateful Operations**
Dependency	Independent per element	Depends on multiple elements
Examples	map, filter, flatMap, peek	sorted, distinct, limit, skip

Parallel Execution	Easy, efficient	Complex, may reduce performance
Memory Requirement	Minimal	Higher (needs to track state)

38. String a1=”ABC”; String a2=”ABC”; String a3=new String(”ABC”);

```
HashMap hm = new HashMap(String,Integer);
Hm.put(a1,2);
Hm.put(a2,1);
Hm.put(a3,3);
Sop(hm.size());
```

Output: {ABC=3} , size=1

39. What is the difference between ExecutorService.submit and ExecutorService.execute in this code in Java?

Ans. A main difference between the submit() and execute() method is that ExecuterService. submit() can return result of computation because it has a return type of Future, but execute() method cannot return anything because it's return type is void.

40. What Causes java.lang.OutOfMemoryError?

Ans. The JVM's memory management scheme sets aside a portion of the heap memory to store newly allocated objects. Any referenced objects remain active in the heap throughout their lifespan (until their reference is closed) and occupy memory. When objects are no longer referenced, they become eligible for the GC to remove them and free up the occupied heap memory.

The Java heap size is determined by two JVM attributes, which can be set when launching Java:

-Xms to set the initial heap size

-Xmx to set the maximum heap size

41. Memory Leak in java with Example? https://www.javatpoint.com/memory-leak-in-java#:~:text=In%20Java%2C%20the%20memory%20leak,that%20may%20lead%20to%20OutOfMemoryError.

Fixing Memory Leak

There are the following solutions to the memory leak problem:

Using JVM Tools: There are many tools available that optimizes the code and show the memory status.

Using Heap Dump: It is a technique that is the solution to the memory leak problem. It is a snapshot of all objects that reside in the memory at a certain time. It also optimizes memory usage in a Java application. It is stored in binary format in hprof

Using Eclipse Memory Leak Warnings: If you are using the Eclipse framework to develop a Java application, eclipse regularly shows the waring and errors whenever it encounters any causes of memory leak.

42. How to solve java.lang.OutOfMemoryError: Java heap space

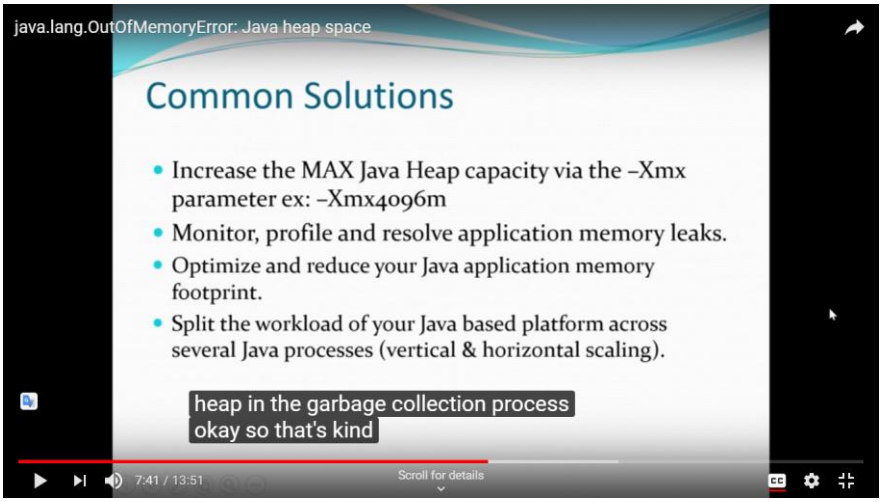
Ans. Everyone in java development faces **java.lang.OutOfMemoryError** now and then, OutOfMemoryError in Java is one problem which is more due to system's limitation (memory) rather than due to programming mistakes in most cases though in certain cases you could have a **memory leak** which causing **OutOfMemoryError**. I have found that even though java.lang.OutOfMemoryError is quite common basic knowledge of its cause and solution is largely unknown among junior developers. Beginners books like Head First Java doesn't teach you much about how to deal with this kind of error. You need real experience dealing with production systems, handling a large number of user sessions to troubleshoot and fix performance issues like running out of memory.

1) An easy way to solve OutOfMemoryError in java is to *increase the maximum heap size* by using JVM options "-Xmx512M", this will immediately solve your OutOfMemoryError. This is my preferred solution when I get OutOfMemoryError in Eclipse, Maven or ANT while building project because based upon size of project you can easily run out of Memory here is **an example of increasing maximum heap size of JVM**, Also its better to keep **-Xmx to -Xms** ration either 1:1 or 1:1.5 if you are setting heap size in your java application

export JVM_ARGS="-Xms1024m -Xmx1024m"

2) The second way to resolve OutOfMemoryError in Java is rather hard and comes when you don't have much memory and even after increase maximum heap size you are still getting java.lang.OutOfMemoryError, in this case, you probably want to profile your application and look for any memory leak.

You can use **Eclipse Memory Analyzer** to examine your heap dump or you can use any profiler like Netbeans or JProbe. This is tough solution and requires some time to analyze and **find memory leaks**.



43. Cohesion in Java? . https://www.geeksforgeeks.org/cohesion-in-java/#:~:text=Cohesion%20in%20Java%20is%20the,of%20that%20class%20is%20more

Ans. Cohesion in Java is the Object-Oriented principle most closely associated with making sure that a class is designed with a single, well-focused purpose. In object-oriented design, cohesion refers all to how a single class is designed.

The advantage of high cohesion is that such classes are much easier to maintain (and less frequently changed) than classes with low cohesion. Another benefit of high cohesion is that classes with a well-focused purpose tend to be more reusable than other classes.

Example: Suppose we have a class that multiplies two numbers, but the same class creates a pop-up window displaying the result. This is an example of a low cohesive class because the window and the multiplication operation don't have much in common. To make it high cohesive, we would have to create a class Display and a class Multiply. The Display will call Multiply's method to get the result and display it. This way to develop a high cohesive solution.

44. How to Handle OutOfMemoryError Exceptions in Java?

Ans. A java.lang.OutOfMemoryError is a runtime error in Java which occurs when the Java Virtual Machine (JVM) is unable to allocate an object due to insufficient space in the Java heap. The Java Garbage Collector (GC) cannot free up the space required for a new object, which causes a java.lang.OutOfMemoryError. This error can also be thrown when the native memory is insufficient to support the loading of a Java class.

45. What is java.lang.OutOfMemoryError in Java?

Ans. OutOfMemoryError in Java is a subclass of java.lang.VirtualMachineError and JVM throws java.lang.OutOfMemoryError when it ran *out of memory in the heap*. OutOfMemoryError in Java can come anytime in heap mostly while you try to create an object and there is not enough space on the heap to allocate that object. Javadoc of OutOfMemoryError is not very informative about this, though.

Two Types of OutOfMemoryError in Java

1) The java.lang.OutOfMemoryError: Java heap space
2) The java.lang.OutOfMemoryError: PermGen space

46. How to handle java.lang.OutOfMemoryError in Java on server?

47. Lock on class level and method level in multi-threading?

48. ClassNotFoundException vs. NoClassDefFoundError?

- **ClassNotFoundException** and **NoClassDefFoundError** occur when a particular class is not found at runtime. However, they occur at different scenarios.
- **ClassNotFoundException** is an exception that occurs when you try to load a class at run time using **Class.forName()** or **loadClass()** methods and mentioned classes are not found in the classpath.
- **NoClassDefFoundError** is an error that occurs when a particular class is present at compile time, but was missing at run time.

ClassNotFoundException	**NoClassDefFoundError**
It is an exception. It is of type java.lang.Exception.	It is an error. It is of type java.lang.Error.
It occurs when an application tries to load a class at run time which is not updated in the classpath.	It occurs when java runtime system doesn't find a class definition, which is present at compile time, but missing at run time.
It is thrown by the application itself. It is thrown by the methods like Class.forName(), loadClass() and findSystemClass().	It is thrown by the Java Runtime System.
It occurs when classpath is not updated with required JAR files.	It occurs when required class definition is missing at runtime.

49. What is DLL?

Ans. A DLL is a library that contains code and data that can be used by more than one program at the same time. For example, in Windows operating systems, the Comdlg32 DLL performs common dialog box related functions.
Dynamic-link library (DLL) is Microsoft's implementation of the shared library concept in the Microsoft Windows and OS/2 operating systems.

50. marker interface and use?

51. Concurrentmap Load factor?

Ans. Creates a new map with the same mappings as the given map. The map is created with a capacity of 1.5 times the number of mappings in the given map or 16 (whichever is greater), and a default load factor (0.75) and concurrencyLevel (16).

52. Why can we create constructor in abstract class and use of it?
Ans.

a. A **constructor** in Java doesn't actually "build" the object, it **is** used to initialize fields. Imagine that your **abstract class** has fields x and y, and that **you** always want them to **be** initialized in a certain way, no matter what actual concrete subclass **is** eventually **created**.
b. Abstract class constructors will be called when its concrete subclass will be instantiated and this is used to initialize variables

53. Java 8 Stream Intermediate And Terminal Operations?

Ans: 1) The main difference between intermediate and terminal operations is that intermediate operations return a stream as a result and terminal operations return non-stream values like primitive or object or collection or may not return anything.

2) As intermediate operations return another stream as a result, they can be chained together to form a pipeline of operations. Terminal operations cannot be chained together.

3) Pipeline of operations may contain any number of intermediate operations, but there has to be only one terminal operation, that too at the end of pipeline.

4) Intermediate operations are lazily loaded. When you call intermediate operations, they are actually not executed. They are just stored in the memory and executed when the terminal operation is called on the stream.

5) As the names suggest, intermediate operations doesn’t give end result. They just transform one stream to another stream. On the other hand, terminal operations give end result.

6) Intermediate Operations : map(), filter(), distinct(), sorted(), limit(), skip()

7) Terminal Operations : forEach(), toArray(), reduce(), collect(), min(), max(), count(), anyMatch(), allMatch(), noneMatch(), findFirst(), findAny()

54. Difference Between Collections And Streams In Java?

1. Collections and Streams, both are conceptually two different things which are used for two different purposes. If the collections are used to store the data then the streams are used to perform operations on that data.
2. Collections are used to store and group the data in a particular data structure like *List*, *Set* or *Map*. But, streams are used to perform complex data processing operations like *filtering*, *matching*, *mapping* etc on stored data such as arrays, collections or I/O resources.
3. You can add to or remove elements from collections. But, you can't add to or remove elements from streams. Stream consumes a source, performs operations on it and returns a result. They don't modify even the source also.

55. External Iteration Vs Internal Iteration

1. The main specialty of Java 8 Streams is that you need not to worry about iteration while using streams. Streams perform iteration internally behind the scene for you. You just have to mention the operations to be performed on a source.
2. Streams are traversable only once. If you traverse the stream once, it is said to be consumed. To traverse it again, you have to get new stream from the source again. But, collections can be traversed multiple times.

56. Why use inheritance in java?

- For Method Overriding (so runtime polymorphism can be achieved).
- For Code Reusability.

57. why default methods in interface and then what the difference between interface and abstract class.

58. what is Annotaion?

59. what is stackoverflow and how to create stackOverflow ?

60. Two classes has main method how to call one main from others?
61. Encapsulation?
62. Java 8 (Functional Interface, Static and default method interface)
63. compare Integer i1=new Integer(10) and Integer i2=new Integer(10)

(i1 == i2) and i1.equals(i2)

64. What is an immutable class?

Ans. Immutable class is a class which once created, it's contents can not be changed.

Immutable objects are the objects whose state can not be changed once constructed. e.g.

String class

65. How to create an immutable class?

Ans. To create an immutable class following steps should be followed:

- Create a final class.
- Set the values of properties using constructor only.
- Make the properties of the class final and private
- Do not provide any setters for these properties.
- If the instance fields include references to mutable objects, don't allow those objects to be changed:
    - Don't provide methods that modify the mutable objects.
    - Don't share references to the mutable objects. Never store references to external, mutable objects passed to the constructor; if necessary, create copies, and store references to the copies. Similarly, create copies of your internal mutable objects when necessary to avoid returning the originals in your methods.

66. I have created an immutable class with a date field. How do I ensure that even the date field is immutable, since even if you make date field final you can still assign a different value to it later? http://java-questions.com/ImmutableClass-interview-questions.html

Lets apply all above rules for immutable classes and make a concrete class implementation for immutable class in Java.

ImmutableClass.java

```
import java.util.Date;
/**
* Always remember that your instance variables will be either mutable or immutable.
* Identify them and return new objects with copied content for all mutable objects.
* Immutable variables can be returned safely without extra effort.
* */
public final class ImmutableClass
{    /**    * Integer class is immutable as it does not provide any setter to change its content    * */
    private final Integer immutableField1;
    /**  * String class is immutable as it also does not provide setter to change its content  * */
    private final String immutableField2;
 /**    * Date class is mutable as it provide setters to change various date/time parts**/
    private final Date mutableField;
```

```
    //Default private constructor will ensure no unplanned construction of class
    private ImmutableClass(Integer fld1, String fld2, Date date)
    {
        this.immutableField1 = fld1;
        this.immutableField2 = fld2;
        this.mutableField = new Date(date.getTime());
    }

    //Factory method to store object creation logic in single place
    public static ImmutableClass createNewInstance(Integer fld1, String fld2, Date date)
    {
        return new ImmutableClass(fld1, fld2, date);
    }
    //Provide no setter methods
    /**
    * Integer class is immutable so we can return the instance variable as it is
    * */
    public Integer getImmutableField1() {
        return immutableField1;
    }
     /**
    * String class is also immutable so we can return the instance variable as it is
    * */
    public String getImmutableField2() {
        return immutableField2;
    }
     /**
    * Date class is mutable so we need a little care here.
    * We should not return the reference of original instance variable.
    * Instead a new Date object, with content copied to it, should be returned.
    * */
    public Date getMutableField() {
        return new Date(mutableField.getTime());
    }
     @Override
    public String toString() {
        return immutableField1 +" - "+ immutableField2 +" - "+ mutableField;
    }
}
```

If the instance fields include references to mutable objects, don't allow those objects to be changed:

a. Don't provide methods that modify the mutable objects.

b. Don't share references to the mutable objects. Never store references to external, mutable objects passed to the constructor; if necessary, create copies, and store references to the copies. Similarly, create copies of your internal mutable objects when necessary to avoid returning the originals in your methods.

67. What are the advantages of immutability?

1. Immutable objects are automatically thread-safe, the overhead caused due to use of synchronisation is avoided.

2. Once created the state of the immutable object can not be changed so there is no possibility of them getting into an inconsistent state.

3. The references to the immutable objects can be easily shared or cached without having to copy or clone them as there state can not be changed ever after construction.

4. The best use of the immutable objects is as the keys of a map.

68. Stream.map() vs Stream.flatMap() in Java 8?

- The function you pass to map() operation returns a single value.
- The function you pass to flatMap() opeartion returns a Stream of value.
- flatMap() is combination of map and flat operation.
- map() is used for transformation only, but flatMap() is used for both transformation and flattening.

Ex:

```
public class Java8Demo {
   public static void main(String args[]) {
      // foods which helps in weight loss
      List<String> loseWeight = new ArrayList<>();
      loseWeight.add("avocados");
      loseWeight.add("beans");
      loseWeight.add("salad");
      loseWeight.add("oats");
      loseWeight.add("broccoli");
      System.out.println("list of String : " + loseWeight);
      // let's use map() method to convert list of weight
      // lose food, which are String to list of ints
      // which are length of each food String
      List listOfInts = loseWeight.stream()
            .map(s -> s.length())
            .collect(Collectors.toList());

      System.out.println("list of ints generate by map(): " + listOfInts);
      // flatMap() example, let's first creat a list of list
      List<List> listOfListOfNumber = new ArrayList<>();
      listOfListOfNumber.add(Arrays.asList(2, 4));
      listOfListOfNumber.add(Arrays.asList(3, 9));
      listOfListOfNumber.add(Arrays.asList(4, 16));
      System.out.println("list of list : " + listOfListOfNumber);
      // let's use flatMap() to flatten this list into
      // list of integers i.e. 2,4,3,9,4,16

      List listOfIntegers = listOfListOfNumber.stream()
            .flatMap( list -> list.stream())
            .collect(Collectors.toList());

      System.out.println("list of numbers generated by flatMap : " + listOfIntegers);

}}
```

Output

list of String : [avocados, beans, salad, oats, broccoli]

list of ints generate by map(): [8, 5, 5, 4, 8]

list of list : [[2, 4], [3, 9], [4, 16]]

list of numbers generated by flatMap : [2, 4, 3, 9, 4, 16]

69. Stack and Heap memory in java?

Java Heap Space: Java Heap space is used by java runtime to allocate memory to Objects and JRE classes. Whenever we create any object, it's always created in the Heap space.

Garbage Collection runs on the heap memory to free the memory used by objects that doesn't have any reference. Any object created in the heap space has global access and can be referenced from anywhere of the application.

Java Stack Memory: Java Stack memory is used for execution of a thread. They contain method specific values that are short-lived and references to other objects in the heap that are getting referred from the method.

Stack memory is always referenced in LIFO (Last-In-First-Out) order. Whenever a method is invoked, a new block is created in the stack memory for the method to hold local primitive values and reference to other objects in the method.

As soon as method ends, the block becomes unused and become available for next method.
Stack memory size is very less compared to Heap memory.

70. Difference between Java Heap Space and Stack Memory

- Based on the above explanations, we can easily conclude following differences between Heap and Stack memory.
- Heap memory is used by all the parts of the application whereas stack memory is used only by one thread of execution.
- Whenever an object is created, it's always stored in the Heap space and stack memory contains the reference to it. Stack memory only contains local primitive variables and reference variables to objects in heap space.
- Objects stored in the heap are globally accessible whereas stack memory can't be accessed by other threads.
- Memory management in stack is done in LIFO manner whereas it's more complex in Heap memory because it's used globally. Heap memory is divided into Young-Generation, Old-Generation etc, more details at Java Garbage Collection.
- Stack memory is short-lived whereas heap memory lives from the start till the end of application execution.
- We can use -Xms and -Xmx JVM option to define the startup size and maximum size of heap memory. We can use -Xss to define the stack memory size.
- When stack memory is full, Java runtime throws java.lang.StackOverFlowError whereas if heap memory is full, it throws java.lang.OutOfMemoryError: Java Heap Space error.

- Stack memory size is very less when compared to Heap memory. Because of simplicity in memory allocation (LIFO), stack memory is very fast when compared to heap memory.

```
public class Memory {

        public static void main(String[] args) { // Line 1
                int i=1; // Line 2
                Object obj = new Object(); // Line 3
                Memory mem = new Memory(); // Line 4
                mem.foo(obj); // Line 5
        } // Line 9

        private void foo(Object param) { // Line 6
                String str = param.toString(); //// Line 7
                System.out.println(str);
        } // Line 8

}
```

Below image shows the Stack and Heap memory with reference to above program and how they are being used to store primitive, Objects and reference variables.

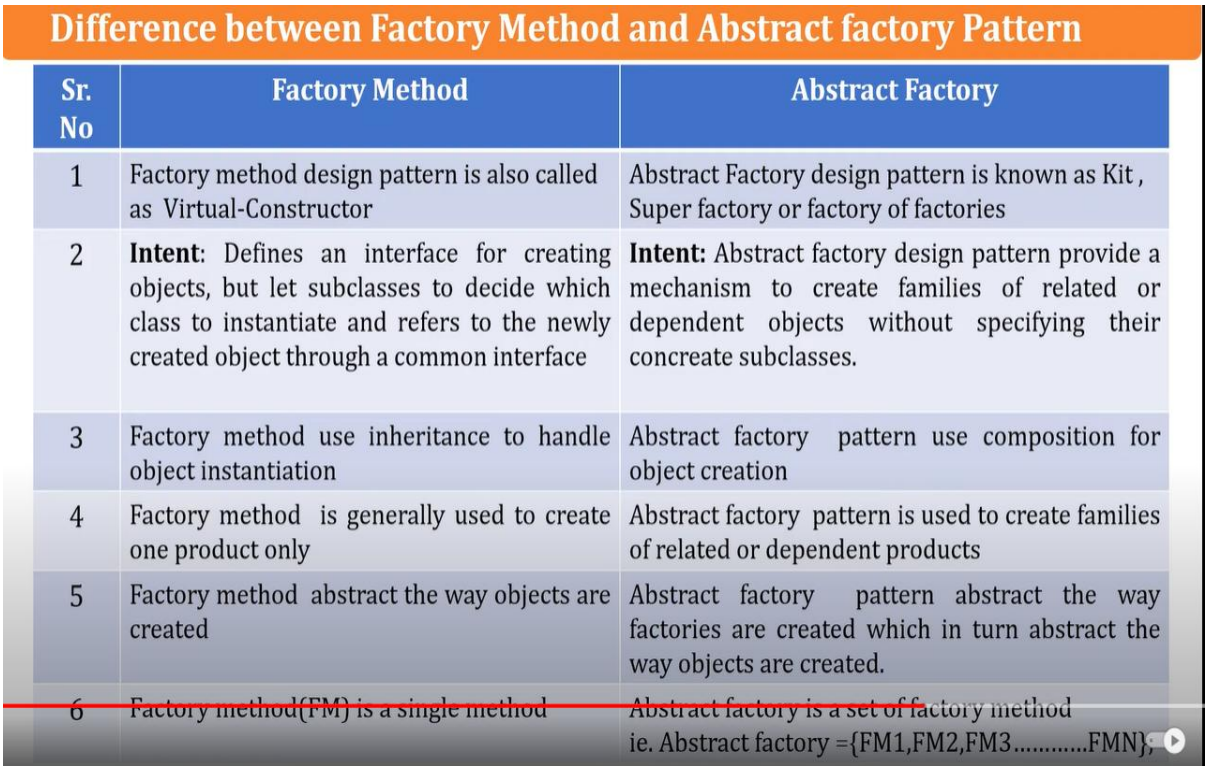
java memory management, java heap space, heap vs stack, java heap, stack vs heap

Let's go through the steps of execution of the program.

As soon as we run the program, it loads all the Runtime classes into the Heap space.
When main() method is found at line 1, Java Runtime creates stack memory to be used by main() method thread.
We are creating primitive local variable at line 2, so it's created and stored in the stack memory of main() method.
Since we are creating an Object in line 3, it's created in Heap memory and stack memory contains the reference for it. Similar process occurs when we create Memory object in line 4.
Now when we call foo() method in line 5, a block in the top of the stack is created to be used by foo() method. Since Java is pass by value, a new reference to Object is created in the foo() stack block in line 6.
A string is created in line 7, it goes in the String Pool in the heap space and a reference is created in the foo() stack space for it.
foo() method is terminated in line 8, at this time memory block allocated for foo() in stack becomes free.
In line 9, main() method terminates and the stack memory created for main() method is destroyed. Also the program ends at this line, hence Java Runtime frees all the memory and end the execution of the program.

Design Pattern:

1. Singleton class?
2. builder design pattern?
3. Builder, Abstact Factory, Factory Design Pattern,Observer)
4. Factory Design And Abstact Factory Pattern



Difference between Factory Method and Abstract factory Pattern

Sr. No	Factory Method	Abstract Factory
1	Factory method design pattern is also called as Virtual-Constructor	Abstract Factory design pattern is known as Kit , Super factory or factory of factories
2	**Intent**: Defines an interface for creating objects, but let subclasses to decide which class to instantiate and refers to the newly created object through a common interface	**Intent:** Abstract factory design pattern provide a mechanism to create families of related or dependent objects without specifying their concreate subclasses.
3	Factory method use inheritance to handle object instantiation	Abstract factory pattern use composition for object creation
4	Factory method is generally used to create one product only	Abstract factory pattern is used to create families of related or dependent products
5	Factory method abstract the way objects are created	Abstract factory pattern abstract the way factories are created which in turn abstract the way objects are created.
6	Factory method(FM) is a single method	Abstract factory is a set of factory method ie. Abstract factory ={FM1,FM2,FM3.....FMN};

Multithread:

1. Atomic variables?

   Atomic variables are classes from the `java.util.concurrent.atomic` package that provide **thread-safe operations without using `synchronized` or locks**.

   Atomic variables provide lock-free, thread-safe updates using CAS (Compare-And-Swap) operations and are ideal for counters, flags, and references shared across multiple threads.

   AtomicInteger
   AtomicLong
   AtomicBoolean
   AtomicReference<T>

   ```
   int count = 0;
   count++;
   ```

   count++ is **not atomic**. It involves:
   1. Read value
   2. Increment
   3. Write value

   Multiple threads can interfere and produce incorrect results

2. **AtomicInteger vs synchronized?**

AtomicInteger	synchronized
Lock-free	Uses lock
Better performance for simple counters	More overhead
Uses CAS	Uses monitor lock
Good for single-variable operations	Good for complex critical sections

3. **Difference between synchronization and volatile keyword?**

Ans. Volatile keyword is not a substitute of synchronized keyword, but it can be used as an alternative in certain cases. There are the following differences are as follows:

Volatile Keyword	**Synchronization Keyword**
Volatile keyword is a field modifier.	Synchronized keyword modifies code blocks and methods.
The thread cannot be blocked for waiting in case of volatile.	Threads can be blocked for waiting in case of synchronized.
It improves thread performance.	Synchronized methods degrade the thread performance.
It synchronizes the value of one variable at a time between thread memory and main memory.	It synchronizes the value of all variables between thread memory and main memory.
Volatile fields are not subject to compiler optimization.	Synchronize is subject to compiler optimization.

4. Race Condition?
5. Thread life cycle implementation.
6. Shutdown hookup?
7. Custom Thread pool?
8. volatile vs AtomicInteger
   (https://www.youtube.com/watch?v=WH5UvQJizH0&ab_channel=DefogTech)

9. JAVA Object/String method overload [duplicate] (**ITC Interview**)

Class1 a=new Class1();

a.method(null) // which method is called

```
public class Class1 {


   public void method(Object obj){
      System.out.println("Object");
   }


   public void method(String str){
      System.out.println("String");
   }


   public static void main(String... arg){
      new Class1().method(null);
   }


}
```

Whenever more than one overloaded methods can be applied to the argument list, the most specific method is used.
In this case either of the methods can be called when passing null, since the "null type" is assignable to both Object and to String. The method that takes String is more specific so it will be picked.

10. Classloader (WildTeck Interview)?

Ans. The **Java ClassLoader** is a part of the **Java Runtime Environment** that dynamically loads Java classes into the **Java Virtual Machine**.

The Java run time system does not need to know about files and file systems because of classloaders.

Java classes aren't loaded into memory all at once, but when required by an application. At this point, the **Java ClassLoader** is called by the **JRE** and these ClassLoaders load classes into memory dynamically.

Types of ClassLoaders in Java - Not all classes are loaded by a single ClassLoader. Depending on the type of class and the path of class, the ClassLoader that loads that particular class is decided. To know the ClassLoader that loads a class the ***getClassLoader()*** method is used. All classes are loaded based on their names and if any of these classes are not found then it returns a **NoClassDefFoundError** or **ClassNotFoundException**.

A Java Classloader is of **three types**:

- **BootStrap ClassLoader:** A Bootstrap Classloader is a Machine code which kickstarts the operation when the JVM calls it. It is not a java class.
- **Extension ClassLoader:** The Extension ClassLoader is a child of Bootstrap ClassLoader and loads the extensions of core java classes from the respective JDK Extension library. It loads files from ***jre/lib/ext*** directory or any other directory pointed by the system property ***java.ext.dirs***.
- **System ClassLoader:** An Application ClassLoader is also known as a System ClassLoader. It loads the Application type classes found in the environment variable ***CLASSPATH, -classpath or -cp command line option***. The Application ClassLoader is a child class of Extension ClassLoader.

11. Autoconfiguration in Spring boot?

12. how to block multiple request from client in rest api in spring boot

Ans. Rate Limiting a Spring API Using Bucket4j

```
<dependency> <groupId>com.github.vladimir-bukhtoyarov</groupId>
<artifactId>bucket4j-core</artifactId> <version>4.10.0</version> </dependency>
@RestController
class AreaCalculationController {

   private final Bucket bucket;

   public AreaCalculationController() {
      Bandwidth limit = Bandwidth.classic(20, Refill.greedy(20, Duration.ofMinutes(1)));
      this.bucket = Bucket4j.builder()
          .addLimit(limit)
          .build();
   }
   //..}
```

13. The Major Difference between User and Daemon Threads:

User Thread	Daemon Thread
User threads are foreground threads.	Daemon threads are background threads.
User threads are high priority threads.	Daemon threads are low priority threads.
Its life independent.	Its life depends on user threads.
JVM wait until user threads to finish their work. It never exit until all user threads finish their work.	The JVM will't wait for daemon threads to finish their work. The JVM will exit as soon as all user threads finish their work.
JVM will not force to user threads for terminating, so JVM will wait for user threads to terminate themselves.	If all user threads have finished their work JVM will force the daemon threads to terminate
User threads are created by the application.	Mostly Daemon threads created by the JVM.
Mainly user threads are designed to do some specific task.	Daemon threads are design as to support the user threads.

Exception

1. What is custom exception and create custom unchecked exception?
2. Exception Handling super & subclass method?
3. difference between throw and throws?
4. Exception Handing in microservice right code
   Suppose A, B C, D, E microservice and exception get in D how to fetch in A

Collection Questions

1. Concurrent Hashmap
2. HashSet and Hashmap
3. Where use comparable and where use comparator?
4. Arralist and vector?
5. how to store object in hashmap as a key?
6. Default Array list size and how to increase?
7. class and interfaces in java?
8. String & string buffer, String Builder
9. Inheritance and how to achieve it and types of Inhiritance?
10. All Oops concepts?
11. how to store object in hashmap as a key?
12. why hashmap contains only single null key?
13. How to create custom arraylist?
14. Treemap internally data structure and working?
15. Hashmap implementation.
16. hashmap internal work?
17. Internal working of hashset?
18. Hashmap And HashTable.
19. Constructor Overring Rule
20. Difference between hashcode() and equals methods?

Note: Java.lang.object has two very important methods defined:

- **public boolean equals(Object obj) and**
- **public int hashCode().**

21. **Where we use Arraylist and where we use linked list?**

    Both LinkedList and ArrayList require O(n) time to find if an element is present or not. However we can do Binary Search on ArrayList if it is sorted and therefore can search in O(Log n) time.

No.	ArrayList	LinkedList
1)	ArrayList uses a dynamic array.	LinkedList uses a doubly linked list.

2)	Manipulation with ArrayList is **slow** because it internally uses an array. If any element is removed from the array, all the bits are shifted in memory.	Manipulation with LinkedList is faster than ArrayList because it uses a doubly linked list, so no bit shifting is required in memory.
3)	ArrayList is better to store and fetch data.	LinkedList is better to manipulate data.
4)	ArrayList provides random access.	LinkedList does not provide random access.
5)	ArrayList takes less memory overhead as it stores only object	LinkedList takes more memory overhead, as it stores the object as well as the address of that object.
6)	An ArrayList class can **act as a list** only because it implements List only.	LinkedList class can **act as a list and queue** both because it implements List and Deque interfaces.

22. Tree map implementation and can we add custom object in tree map?
23. **Find Maximum length of a binary tree.**
24. **TreeSet< Student > s = new TreeSet<Student>();**

    TreeSet<SttringBuffer> s = new TreeSet<StringBuffer>();

    Ans

    1. An object is said to be comparable if and only if the corresponding class implements **Comparable interface**.
    2. **String** class and all **Wrapper** classes already implements Comparable interface but StringBuffer class doesn't implements Comparable interface.Hence we got *ClassCastException* in the above example.

25. **Difference between Hashmap and Concurrent Hashmap?**

    HashMap is the Class which is under Traditional Collection and ConcurrentHashMap is a Class which is under Concurrent Collections, apart from this there are various differences between them which are:

- HashMap is non-Synchronized in nature i.e. HashMap is not Thread-safe whereas ConcurrentHashMap is Thread-safe in nature.
- HashMap performance is relatively high because it is non-synchronized in nature and any number of threads can perform simultaneously. But ConcurrentHashMap performance is low sometimes because sometimes Threads are required to wait on ConcurrentHashMap.
- While one thread is Iterating the HashMap object, if other thread try to add/modify the contents of Object then we will get Run-time exception saying **ConcurrentModificationException**.Whereas In ConcurrentHashMap we wont get any exception while performing any modification at the time of Iteration.

26. Which collection classes are synchronized or thread-safe?

    ConcurrentHashMap, SynchronizedHashMap, Stack, Properties, Vector,Hashtable

    BlockingQueue(I), ConcurrentMap, ConcurrentNavigableMap

27. **Type of classes in java?**

    Ans:
    a. Static Class.
    b. POJO Class
    c. Concrete Class
    d. Abstract Class
    e. Final Class
    f. Inner Class(Nested Inner class,Method Local inner classes, Anonymous inner classes, Static nested classes)

28. how to insert multiple same keys on hashmap?
29. Create Student class object in hashmap?

Spring

1. Spring Boot.
2. @ComponentScan
3. How Spring Boot Application works internally?

    Ans. The application starts using the "main method" which calls the "run" method (static method of SpringApplication class). From the run method, the main application context

kicks off which searches for the classes annotated with @Configuration and initializes all the declared beans in those configuration classes. Based on the scope of those beans, stores those beans in JVM, specifically in a space inside JVM which is known as IOC container. After the creation of all the beans, automatically configures the dispatcher servlet and registers the default handler mappings, messageConverts, etc.

4. Spring Security
5. In Maven – artifact Id, version , snapshot?
6. **how to process the image in spring from jsp to controller?**
7. Spring modules?
8. Scope of bean?
9. what is version?
10. **why tree set does not contain null value(<u>Nuclilios</u>)?**

    Ans. Adding null values to a tree set
    TreeSet adds elements to it according to their natural order. This internally compares the elements with each other using the compareTo (or compare) method.

    If you try to compare any object with a null value using one of these methods, a NullPointerException will be thrown.

    Therefore, if you try to add null values to a TreeSet it generates a NullPointerException at the run time.

11. DI and IOC?
12. How to enable security in Spring? Difference between Basic authentication and Form based ?
13. @ModelAttribute & @path variable?
14. Spring annotations
15. Spring boot vs spring framework?
16. blocking and non blocking request?(synchronized & not Syncronized)
17. Hashcode and equals difference?
18. Spring boot questions.
19. Why use setting.xml in spring boot.(https://maven.apache.org/settings.html)
20. Spring Initializer?
21. Actuator and CLI?
22. How activate login in spring boot and LOGGING level in Spring Boot?

    (Logging level describe in application properties . levels are

    {WARN,DEBUG,FATAL,MESSAGE})
23. where you use Spring boot in you project?

24. how to get value from url in rest controller?

25. how to process image in spring from jsp to controller?

26. what is java action tags and difference between include and directive tags?

27. Spring Bean Configuration: How many type to create Spring bean?

Spring Framework provides three ways to configure beans to be used in the application.

1. **Annotation Based Configuration** – By using @Service or @Component annotations. Scope details can be provided with @Scope annotation.
2. **XML Based Configuration** – By creating Spring Configuration XML file to configure the beans. If you are using Spring MVC framework, the xml based configuration can be loaded automatically by writing some boiler plate code in web.xml file.
3. **Java Based Configuration** – Starting from Spring 3.0, we can configure Spring beans using java programs. Some important annotations used for java based configuration are @Configuration, @ComponentScan and @Bean.

28. @RequestParam & @path variable?

a. Even though both @RequestParam and @PathVariable is used to extract values from the HTTP request, there is a subtle difference between them, which makes them a useful question from an interview and spring certification point of view.

b. We'll examine the subtle difference between @RequestParam and @PathVaraible in this article. As the name suggests, @RequestParam is used to get the request parameters from URL, also known as query parameters, while @PathVariable extracts values from URI.

c. For example, if the incoming HTTP request to retrieve a book on topic "Java" is http://localhost:8080/shop/order/1001/receipts?date=12-05-2017, then you can use the @RequestParam annotation to retrieve the query parameter date and you can use @PathVariable to extract the orderId i.e. "1001" as shown below:

```
@RequestMapping(value="/order/{orderId}/receipts", method =
RequestMethod.GET)
public List listUsersInvoices(@PathVariable("orderId") int order,
 @RequestParam(value = "date", required = false) Date dateOrNull) {
...
}
```

The required=false denotes that the query parameter can be optional, but the URL must have the same URI.

29. Spring Initilizr

30. Spring Boot IDEs

31. Spring Boot Framework has mainly four major Components.

- Spring Boot Starters
- Spring Boot AutoConfigurator
- Spring Boot CLI
- Spring Boot Actuator

32. How does a spring container work? // Spring Study.docs

33. How many type of container in spring mvc?

34. Implement Logger in Spring(using slf4j.jar, log4j) (@Slf4j)

35. Common Versioning Strategies in Spring Boot

- **URI Path Versioning- GET /api/v1/users**
- **Query Parameter Versioning : GET /api/users?version=1**
- **Header Versioning : GET /api/usersX-API-VERSION=1**
- **Content Negotiation : Accept: application/vnd.company.app-v1+json**

Code Examples

1. URI Path Versioning

```
@RestController
@RequestMapping("/api/v1/users")
public class UserControllerV1 {
   @GetMapping
   public List<User> getUsersV1() {
      return List.of(new User("John Doe"));
   }
}

@RestController
@RequestMapping("/api/v2/users")
public class UserControllerV2 {
   @GetMapping
   public List<UserV2> getUsersV2() {
      return List.of(new UserV2("John", "Doe"));
   }
}
```

2. Query Parameter Versioning

```
@RestController
@RequestMapping("/api/users")
public class UserController {
   @GetMapping(params = "version=1")
   public User getUserV1() { return new User("John Doe"); }

   @GetMapping(params = "version=2")
   public UserV2 getUserV2() { return new UserV2("John", "Doe"); }
}
```

3. Header Versioning

```
@RestController
@RequestMapping("/api/users")
public class UserHeaderController {
    @GetMapping(headers = "X-API-VERSION=1")
    public User getUserHeaderV1() { return new User("John Doe"); }

    @GetMapping(headers = "X-API-VERSION=2")
    public UserV2 getUserHeaderV2() { return new UserV2("John", "Doe"); }
}
```

4. Content Negotiation

```
@RestController
@RequestMapping("/api/users")
public class UserProducesController {
    @GetMapping(produces = "application/vnd.company.app-v1+json")
    public User getUserV1() { return new User("John Doe"); }

    @GetMapping(produces = "application/vnd.company.app-v2+json")
    public UserV2 getUserV2() { return new UserV2("John", "Doe"); }
}
```

⚠️ Best Practices

- Start with URI path versioning (simplest and most widely used).

36. Application context and bean factory?

37. what is Autowiring?

38. @Controller vs RestController

Ans: some important differences between these two annotations.
The job of @Controller is to create a Map of model object and find a view
but @RestController simply return the object and object data is directly written into HTTP response as JSON or XML.
This can also be done with traditional @Controller and use @ResponseBody annotation but since this is the default behavior of RESTful Web services, Spring introduced @RestController which combined the behavior of @Controller and @ResponseBody together.

- The @Controller is a common annotation which is used to mark a class as Spring MVC Controller while @RestController is a special controller used in RESTFul web services and the equivalent of @Controller + @ResponseBody.
- The **@RestController** is relatively new, added only on Spring 4.0 but @Controller is an old annotation, exists since Spring started supporting annotation, officially it was added on Spring 2.5 version.
- The @Controller annotation indicates that the class is a "Controller" like a web controller while @RestController annotation indicates that the class is a controller where @RequestMapping methods assume **@ResponseBody** semantics by default i.e. servicing REST API.

```
@Controller
@ResponseBody
public class MVCController {
   .. your logic
}

@RestController
public class RestFulController {
  .... your logic
}
```

39. Difference b/w CrudRepository and @JPARepository

40. Why should we use Spring Boot Framework?

Ans:

- The dependency injection approach is used in Spring Boot.
- It contains powerful database transaction management capabilities.
- It simplifies integration with other Java frameworks like JPA/Hibernate ORM, Struts, etc.
- It reduces the cost and development time of the application

41. Patch and Put difference?

42. @EnableAutoConfiguration vs @SpringBootApplication

Even though both **@SpringBootApplication** and **@EnableAutoConfiguration** can be used to enable the **auto-configuration feature of Spring Boot** there is a subtle difference between them. The @SpringBootApplication does much more than what @EnableAutoConfiguration do. It's actually a combination of three annotations: **@Configuration** which is used in Java-based configuration on Spring framework, **@ComponentScan** to enable component scanning of components you write e.g. @Controller classes, and **@EnableAutoConfgiuration** itself, which is used to enable auto-configuration in Spring Boot application. Spring Boot designers realize that these three annotations are frequently used together so they bundled them into @SpringBootApplicaiton. Now, instead of three annotations you just need to specify one annotation on your Main class.

The @EnableAutoConfiguration annotation allows you to **selectively exclude certain classes** from auto-configuration using exclude attribute as shown below:

```
@Configuration

@EnableAutoConfiguration(exclude={DataSourceAutoConfiguration.class})

public class MyConfiguration {

  //.. Java code
```

}

The @EnableAutoConfiguration annotations enable auto-configuration features of Spring Boot which configures modules based on the presence of certain classes on the classpath. For example, if Thymeleaf JAR is present in classpath and Spring MVC is enabled e.g. using spring-boot-web-starter package then it can automatically configure template resolver and view resolver for you.

The @EnableAutoConfiguration annotation is based on @Conditional annotation of Spring 4.0 which enables conditional configuration.

In case of auto-configuration, manually declared beans can override beans automatically created by auto-configuration feature. This is achieved by using @ConditionalOnMissingBean of Spring 4.0

Hibernate

1. **How its work @Transactional – Isolation and Propogation**
2. difference persistent states of hibernate?
3. Aggregation and composition are the part of Association?
   https://www.youtube.com/watch?v=9nRblRcb35Y&t=585s
   Ans. Association is the general relationship between objects. Aggregation and Composition are specialized forms of Association. Aggregation represents a weak "has-a" relationship where objects can exist independently, while Composition represents a strong "has-a" relationship where the child object's lifecycle depends on the parent.

   Association (Has-A Relationship)
   A general relationship between two classes.
   Java
   class Teacher {
   }

   class Student {
   Teacher teacher;
   }
   A student is associated with a teacher.

   Aggregation (Weak Has-A)

The child object can exist independently of the parent.

Example:

Java

```
class Department {
}

class Employee {
Department department;
}
```

- Employee has a Department.
- If Department is deleted, Employee can still exist.

Example: College ↔ Student

Students can exist even if the college is closed.

Composition (Strong Has-A)

The child object cannot exist without the parent.

Java

```
class Engine {
}

class Car {
private Engine engine = new Engine();
}
```

- Engine belongs to Car.
- If Car is destroyed, Engine is also destroyed.

Example: House ↔ Room

A room cannot exist without a house.

4. union and union All?
5. Composite key?
6. why use Serial-version-id and difference between auto generated and manually created id?
7. Joins in hibernate?
8. how to filter odd no, of arraylist?
9. Second level cahche in hibernate and how to implement?

10. N+1 problem in Hibernate?

The **N+1 query problem** occurs when Hibernate executes:

- **1 query** to fetch parent records
- **N additional queries** to fetch child records for each parent

This results in **N+1 database queries**, causing performance issues.

11. how to stop full rollback while running multiple operation in @Transactional operation?

To avoid full rollback in a `@Transactional` method, use `Propagation.REQUIRES_NEW` for operations that must commit independently, or configure `noRollbackFor` for specific exceptions. `REQUIRES_NEW` is the most common production approach for partial commit scenarios such as audit logging, notifications, and batch processing.

12. There are two classes A and b, b extends A how many table will created?

Ans. If no inheritance strategy is specified, JPA/Hibernate defaults to **SINGLE_TABLE**, so typically **1 table is created**.

Number of tables depends on the inheritance strategy.

1. @Inheritance(strategy = InheritanceType.SINGLE_TABLE) = Single Table
2. @Inheritance(strategy = InheritanceType.TABLE_PER_CLASS) = Two tables
3. @Inheritance(strategy = InheritanceType.JOINED) = Two tables

```
@Entity
class A {
  @Id
  private Long id;
}
@Entity
class B extends A {
  private String name;
}
```

13. how to composite key create in Hibernate?

Ans: By @EmbeddedId,@Embeddable

Rules to create class with composite key

- class must be a public class.
- must have a default constructor.
- Must implement serialize interface.
- Class not require primary key its own.
- Class must have equals and hashcode method.

14. Propagation in @Transactional

15. @EnableTransaction

16. what is composite key?

- The composite primary key in the database is the group of columns whose value together make unique value.
- If unique key is contained in single column than @Id is used and when the primary key consists of multiple column, these columns need to be group into a different object.
- To group multiple column in a class, some rules have to be followed.

MicroService

1. **Microservice Design Pattern architecture**
   https://www.edureka.co/blog/microservices-design-patterns.
   - API Gateway Design Pattern
   - Circuit Breaker Pattern
   - Hypothtical pattern
   - Saga Design pattern
2. Transaction Management in Microservice application?
3. what is microservices and advantages and disadvantages?
4. How to communicate microservice?
5. Datastructure use in microservice or spring boot?
6. Difference between Monolithic & Micro Service Application?
7. **Microservices CQRS Design Pattern?**

Web

1. Difference between http put and http post**?**
2. **REST and Soap?**
3. **Is WebClient asynchronous or synchronous?**

   Unlike RestTemplate, WebClient is asynchronous and non-blocking. Similar to Spring WebFlux, it enables reactive programming, and is based on an event-driven structure.

4. **Is REST API synchronous or asynchronous?**
   Synchronous calls are the most common method of making requests in a REST API. In synchronous calls, the client application sends a request to the server and waits for a response before proceeding to the next step. This means that the client application is blocked until it receives a response from the server.

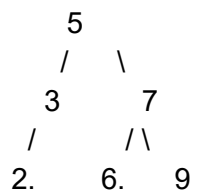
5. **Difference 401 and 403?**
   a. 401 : *Unauthorized* response should be used for missing or bad authentication,
   b. 403: The server understood the request, but is refusing to fulfill it.?
   c. 401 is Authentication error, 403 is Authorization error.

Database:

1. Normalization and denormalization?
2. **ROLLBACK and ROLLBACK TO statement.**
   Ans. There is a considerable difference between the ROLLBACK and ROLLBACK TO statement. When you apply the ROLLBACK command a transaction is completely undone where all the blocks are released. As far as ROLLBACK TO command is concerned, the transaction is undone but till a SAVEPOINT. Thus, the transaction remains live and active even after the command is implemented.

Data Structure

1. Binary search tree



   If you have a pointer at 2 than return the next node on the same level i.e.., 6.

2. Reverse a linklist in java implementation?
3. Find Maximum length of a binary tree?
4. mergesort [_2_|_3_|_5_|_2_|_6__7] Algo

==========**NIIT**==========

- Difference between get and load?
- Difference b/w servlet and filter?
- How to filter ips based on user and password?
- Describe current project?

Angular Question:

1. Observation
2. Difference between Attribute Directive and Structure Directive.
3. What is module?
4. Lazy load.
5. .remove and .map in jquery?
6. Document.ready{} in jquery.
7. JQuery chaning
8. Rsjx
9. Indexion in html 5
10. Singleton in angular
11. Lazyloading in angular
12. Angular Component and service and module
13. What is node js?
14. @Component in angular
15. What is back-trace in angular

Javascript

1. Difference Between =, == And === In JavaScript

- = is used for assigning values to a variable in JavaScript.
- == is used for comparison between two variables irrespective of the datatype of variable.
- === is used for comparision between two variables but this will check strict type, which means it will check datatype and compare two values.

```
<script type="text/javascript">
   function Comparision() {
      var number = 100;  // Here number variable assigned using =
      debugger;

      if (number == 100)  // Here Comparision between two values using ==. This will not
check datatype irrespective of datatype it will                    do comparision
         $("#lblMessage").text("Both are equal");
      else
         $("#lblMessage").text("Both are not equal");

     if(number == "100")  //Here Comparision between two values using ==. This will not
check datatype irrespective of datatype it will                    do comparision
         $("#lblMessage1").text("Both are equal");
      else
        $("#lblMessage1").text("Both are not equal");
   }
</script>

script type="text/javascript">
   function Comparision() {
      var number = 100;  // Here number variable assigned using =
      debugger;

      if (number === 100)  // Here Comparision between two values using ==. This
will not check datatype irrespective of datatype it
                  will do comparision
         $("#lblMessage").text("Both are equal");
      else
         $("#lblMessage").text("Both are not equal");

      if (number === "100")  // Here Comparision between two values using ==. This
will not check datatype irrespective of datatype it                    will do
comparision
         $("#lblMessage1").text("Both are equal");
      else
        $("#lblMessage1").text("Both are not equal");
   }
</script>
```

2. **Doc-type in html?**

The <!DOCTYPE> declaration must be the very first thing in your HTML document, before the <html> tag.

The <!DOCTYPE> declaration is not an HTML tag; it is an instruction to the web browser about what version of HTML the page is written in.

In HTML 4.01, the <!DOCTYPE> declaration refers to a DTD, because HTML 4.01 was based on SGML. The DTD specifies the rules for the markup language, so that the browsers render the content correctly.

HTML5 is not based on SGML, and therefore does not require a reference to a DTD.

Kafka

1. Replication use in Kafka?
2. **Difference between offset commit and offset reset?**

   Offset Commit → Saves current reading position.

   Offset Reset → Changes the reading position to a different offset.

3. **What is offset reset in Kafka?**

   Offset reset is the process of changing a consumer group's reading position in a topic partition. It allows consumers to reprocess old messages, skip messages, or start consumption from a specific offset, timestamp, earliest, or latest position.

4. what is offset in kafka and Reset Offset Technique in Kafka?

Offset is the position of a consumer in a partition. Resetting offsets means changing where a consumer starts reading messages.

Why Reset Offsets?

Common use cases:

1. Reprocess old messages
2. Skip problematic messages
3. Recover from consumer failures

4. Replay data for debugging/testing

1. **`auto.offset.reset` (application.properties file)**
   - Used when no committed offset exists for the consumer group.
     `auto.offset.reset=earliest`
   - Read from beginning of partition.
     `auto.offset.reset=latest`
   - `Read only new messages -> Throw exception if offset doesn't exist.`
     `auto.offset.reset=none`

2. **Reset Consumer Group Offset Using Kafka CLI**

```
kafka-consumer-groups.sh \
--bootstrap-server localhost:9092 \
--group order-group \
--describe
```

5. **Kafka – Consumer Lag (https://www.redpanda.com/guides/kafka-performance-kafka-consumer-lag)**

Ans: Kafka consumer lag is the difference between the last message produced by the producer and the offset committed by the consumer group. It represents the consumer processing delay.

Kafka consumer lag concept	Description
What is Kafka consumer lag?	Kafka consumer lag is the difference between the last offset stored by the broker and the last committed offset for that partition.
Reasons for Kafka consumer lag	Four common reasons for consumer lag are (1) Incoming traffic surges, (2) Data skew in partitions, (3) Slow processing jobs, and (4) Errors in code and pipeline components
Monitoring Kafka consumer lag	Teams can monitor Kafka consumer lag with the consumer group script, Burrow (a Kafka monitoring companion), or generic monitoring tools and exporters.

Strategies for addressing Kafka consumer lag	The best strategy to address Kafka consumer lag will vary depending on the underlying cause. Processing logic optimizations, partition count modifications, rate limiting, and configuration tuning are four common strategies to address consumer lag.

Extra Question

1. cycling barrier?
2. Rechardson model of modularity
3. counter latch?
4. Semaphore
5. CMS Algorithm

6. **Deep and shallow cloning?**
   Ans. Object.clone() performs Shallow Copy by default.
   Shallow cloning copies the object but shares referenced objects, whereas deep cloning copies both the object and all nested referenced objects, making them completely independent.

   OR

   Shallow cloning copies only the main object and shares child objects. Deep cloning copies both the main object and all referenced child objects, making them completely independent.
   Real-Life Example
   Think of a house:

- **Shallow Clone**: Two people get copies of the house key for the same house.
- **Deep Clone**: A completely new house is built with its own key.

   Interview One-Liner:
   Shallow Copy = shared references.
   Deep Copy = copied references.

7. g1 collector
8. Survival space and hidden space?
9. young generation and old generation?
10. Configuration external jar class in project.
    Ans: (@SpringBootApplication(scanBasePackages = {"com.jkoder.shopping", "com.commons.shopping"}))
11. Generic class?
12. Messeging Queue like Rabbit?
13. upper bound and lower bound?
14. how much data stored in session and local storage

15. common vulnerability issue – use new version
16. micro service best practice?
17. what is hook in angular
18. ngOnInit() and init()
19. how to transfer data between iframes
20. Producer-Consumer problem ?
21. Messeging Queue like Rabbit
22. **Solid Principle in java**

    In Java, **SOLID principles** are an object-oriented approach that are applied to software structure design. These five principles have changed the world of object-oriented programming, and also changed the way of writing software. It also ensures that the software is modular, easy to understand, debug, and refactor.
    The word SOLID acronym for:

    - Single Responsibility Principle (SRP)
    - Open-Closed Principle (OCP)
    - Liskov Substitution Principle (LSP)
    - Interface Segregation Principle (ISP)
    - Dependency Inversion Principle (DIP)

Junit

1. **The Difference Between JUnit and Mockito?**

   While JUnit focuses on testing individual units of code, Mockito specializes in managing dependencies and mocking external interactions. By integrating both JUnit and Mockito, developers can create more robust, efficient, and comprehensive test suites that cover a wide range of testing scenarios.

2. **What is Apache Camel?**

   Apache Camel ™ is a versatile open-source integration framework based on known Enterprise Integration Patterns.

   Camel empowers you to define routing and mediation rules in a variety of domain-specific languages (DSL, such as Java, XML, Groovy, Kotlin, and YAML). This means you get smart completion of routing rules in your IDE, whether in a Java or XML editor.

   Apache Camel uses URIs to work directly with any kind of transport or messaging model such as HTTP, Kafka, JMS, JBI, SCA, MINA or CXF, as well as pluggable Components and Data Format options. Apache Camel is a small library with minimal dependencies for easy embedding in any Java application. Apache Camel lets

you work with the same API regardless which kind of transport is used — so learn the API once and you can interact with all the Components provided out-of-box.

Apache Camel provides support for Bean Binding and seamless integration with popular frameworks such as Spring.

Programs

1. Count pallendrom string.
2. Prime no.
3. Find max occuring integer value from array?
4. Find or count Pallindrom String abcab ? output ab 2 two times
5. Get third even number from list using java 8?
6. Given a Student Class contains name, class, Math Marks, Computer Marks, physics Marks, chemistry Marks.
   And Create Two List 1st List Contains name, class, Math Marks and Computer Marks.
   And Second List Contains name, class, physics Marks, chemistry Marks.
   Student has differentiate by name and class.
   1st list contains Records of Students (S1, S2, S3, S4)
   2ND list contains Records of Students (S3,S4,S5.S6)
   So create 3rd list that contains only s1, s2, s3, s4, s5, s6 students' records and merge records of subject marks.

7. how to run multithread to print value 111222333444555?
8. Hashmap With Employee Object and Sort by salary?
   HashMap<Integer, Employee >

AWS

1. How to use different region Regions database in aws?
2. AWS RDS Read Replica across AWS Region
3. Copying AMI across AWS regions, Copying Amazon EC2 AMI to different region, AMI Transfer
4. Move AWS EC2 Instance to Another AWS Region.
5. How to use AWS Transit Gateway in 2 different regions in single aws account?
6. AWS Application Load Balancer (2023) Step-by-Step Tutorial with Target Groups and EC2, #AWS #ALB
7. **Q: Aws lambda limitations?**

   Ans: AWS Lambda, a serverless computing product, has several limitations, including:

Memory: The memory limit is 3 GB.

Timeout: The maximum execution timeout for a function is 15 minutes.

Deployment package size: The default deployment package size is 50 MB, and the package size with layers is 250 MB.

Concurrent execution: The default limit is 1,000 concurrent executions.

Payload size: The request and response (synchronous calls) body payload size can be up to 6 MB, and the event request (asynchronous calls) body can be up to 128 KB.

Disk space: The disk space (ephemeral) is limited to 512 MB.

Function code storage: The limit is 75 MB.

Scaling: Lambda may not match a traditional web server for speed of execution and ability to scale.

Function name: If you specify only the function name, it is limited to 64 characters in length.

Other limitations include cold starts, unsupported languages, limited networking capabilities, and debugging capabilities limits.

Kubernetes

1. Check log in Kubernetes

Ans. To check logs in Kubernetes, use these commands:

kubectl logs <pod-name> -

kubectl logs nginx-pod

View Logs from a Specific Namespace

kubectl logs <pod-name> -n <namespace>

kubectl logs nginx-pod -n production

Using DesktopUi

1. Lens (Recommended Desktop UI): For production environments, **Lens** is often the easiest and most developer-friendly option.
2. Open Rancher UI.

Amazon Interview Required

- **Data structures you should know:** Arrays, Stacks, Queues, Linked lists, Trees, Graphs, Hash tables
- **Algorithms you should know:** Breadth First Search, Depth First Search, Binary Search, Quicksort, Merge sort, Dynamic programming, Divide and Conquer